

Spiegazione del codice e del suo utilizzo

Il codice è organizzato in vari file, ciascuno dei quali svolge una funzione specifica.

1 preprocessing.py

Chiamato una sola volta per costruire i dataset

preprocessing.py legge i file “raw” del dataset HCP e li organizza creando 3 dizionari (uno per ogni task) della forma {id_paziente: serie_temporale_paziente}. Inoltre scarta tutti i pazienti che non hanno tutte e 3 le task registrate, così che alla fine ogni paziente ha una run in ogni task. Infine, salva i tre dizionari in file .npy.

2 dataset.py

Non chiamato esplicitamente

dataset.py contiene tutte le utilities necessarie per maneggiare i dati. In particolare, contiene la funzione split_data, che legge i dizionari relativi alle task di interesse, separa i pazienti in train/val/test, normalizza i dati con le statistiche del training set, crea i Dataset e DataLoader.

Sia i Dataset che i DataLoader sono classi custom, create per le esigenze del progetto. Il TimeSeriesDataset era già presente nel codice originale, noi abbiamo risolto un piccolo bug che impediva di caricare l’ultima (window, target) e abbiamo aggiunto la possibilità di concatenare velocità e accelerazione e fare data augmentation. Sono tutti parametri specificabili nel config.

Il DataLoader è GPUBatchLoader, che permette di caricare tutto il dataset su GPU e estrarre un batch con una singola chiamata vettorizzata. Velocizza di molto il training, dato che il dataset è composto da molti dati di piccole dimensioni (tante chiamate con loop python altrimenti).

L’ultima classe in dataset.py è MultiTaskLoader, che permette di creare un DataLoader con vari task simultaneamente. Il problema è che GPUBatchLoader richiede un singolo T (lunghezza della time series), ma i vari task hanno T diversi. MultiTaskLoader mischia i GPUBatchLoader relativi a task diverse in modo da avere un singolo DataLoader.

3 engine.py

Non chiamato esplicitamente

engine.py contiene la funzione fit() necessaria per allenare i vari modelli. Viene usata anche in fase di finetuning. È un classico training loop + validation loop con la possibilità di usare la loss “normale” o la loss per il finetuning, permette di aggiungere il parametro gamma (jerk penalty) e calcola ad ogni epoca loss e fd gain per train/val, che printa a schermo.

4 train.py

Uso di train.py: python train.py <train_config>.yaml

train.py è l'intero script di training, che guarda il config di training, ne estrae i parametri, costruisce il modello e i dataset train/val/test, salva il miglior modello con il suo config, μ e σ , id dei pazienti su cui ha trainato/validato/testato, e anche la distribuzione delle σ predette sul validation set (questo serve per poi calcolare un percentile per usare il threshold mechanism, che però non usiamo mai perché inefficace).

5 metrics.py

Non chiamato esplicitamente

metrics.py è un piccolo file che contiene alcune funzioni per metriche (fd(), fd_gain(), jerk_penalty()) ma principalmente la funzione evaluate() che è un semplice evaluation loop, in cui però vengono salvate tutte le quantità di interesse per poter analizzare i risultati. In particolare, su ogni (window, target) del test set, si ricava: predizione, σ , target, baseline, fd_pred, fd_base, z ($y - \mu / \sigma$). In più, per ogni frame viene salvato il paziente di appartenenza, così poi da poter fare analisi paziente per paziente (aggregando tutti i frame di un paziente). evaluate() permette anche di applicare la threshold in σ e la corruzione con rumore

6 evaluate.py

Uso di evaluate.py: python evaluate.py <checkpoint>.pth

evaluate.py contiene il codice per evaluation sul test set (sfrutta evaluate() di metrics.py) e l'analisi dei dati + visualizzazioni. In particolare, evaluate.py chiama evaluate() per ogni task di testing, aggrega tutti i risultati e poi fa una serie di visualizzazioni.

7 finetune.py

Uso di finetune.py: python finetune.py <finetune_config>.yaml

finetune.py contiene il codice per finetunare un certo modello. Legge il config, carica il modello da finetunare a partire dal pre-trained checkpoint. Per ogni paziente nel test set del modello originale, prende ogni time series e la divide in train/val/test cronologicamente senza leakage. Valuta prima le performance e l'accuracy del modello pre trained, poi separa i parametri del modello in logvar-related: frozen, mean-head-related: learning rate normale, e body: learning rate piccolo. Poi manda fit() con i vari iperparametri e finetuning loss (MSE + L2SP). Infine, salva un csv di riassunto per ogni paziente con varie metriche prima e dopo il finetuning (sugli stessi frame di test) e poi un .npz con tutti i valori di pred, σ , target, baseline, fd_pred, fd_base, z sui frame di test.

8 finetune_plots.py

Dopo finetune.py, per ottenere le visualizzazioni è sufficiente eseguire finetune_plots.py: python finetune_plots.py

9 benchmark.py

Uso benchmark.py: python benchmark.py <folder>

benchmark.py viene usato per stilare la classifica tra i checkpoint salvati in una cartella. Valuta numerose metriche, inclusa noise robustness (che richiede un certo numero di evaluation con vari livelli di noise), e salva un csv con la classifica (ordinata per FD_gain).

10 distill.py

Uso distill.py: python distill.py <distill_config>.yaml

distill.py contiene il codice relativo alla distillazione. Legge il config dove è inserito il checkpoint relativo al teacher e il config per lo student. Assume che la sequence length del teacher sia la stessa dello student. Nel config sono specificati anche 3 parametri della distillazione vera e propria: il peso dato alle feature in output (KL divergence di (mean, var)), il peso dato alle feature in input (MSE nello spazio latente) e la temperatura del teacher per la KL divergence (appiattimento della sigma del teacher). La distillation avviene sullo stesso training set del teacher così da evitare leakage quando si va in fase di testing. Per ottenere le feature dello spazio latente, ci sono degli hook che raccolgono le attivazioni prima delle mean e variance head così da poter applicare MSE ai due spazi latenti di teacher e student (previa proiezione tra i due spazi). Infine salva il checkpoint, eseguibile da evaluate.py

11 routing.py

Uso routing.py: python routing.py

routing.py contiene il codice relativo all'ensembling dei migliori modelli. All'interno sono linkati i checkpoint degli 'esperti', e un piccolo MLP è addestrato a partire dalle previsioni (mean e var) di tutti gli esperti (e della baseline) a predire dei pesi (softmax) per combinare le previsioni dei vari modelli per massimizzare l'FD_gain. Finito il training (sull'85% del validation set, 15% validation lasciato come effettiva validation), viene valutato il routing attivo contro una media fissa contro la combinazione fissa data dai pesi assegnati in media ai vari modelli.

12 models.py

Non chiamato esplicitamente

models.py contiene tutte le architetture, alcune utils per contare i FLOPs e la latenza, e il 'model registry' che serve per associare un nome come "gru" all'architettura quando si specifica il tipo di architettura nel config.